

# Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

## AppBar, NavDrawer y FAB



Universidad  
Rey Juan Carlos

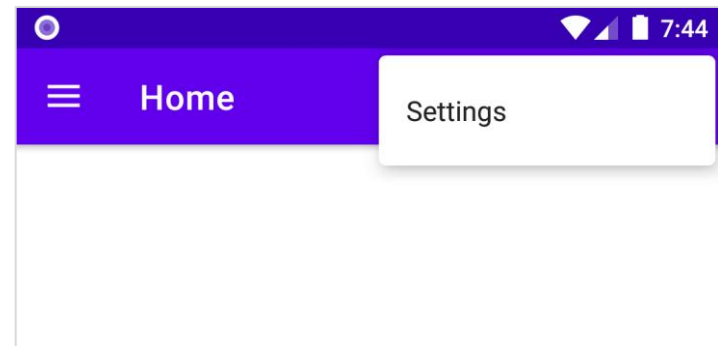
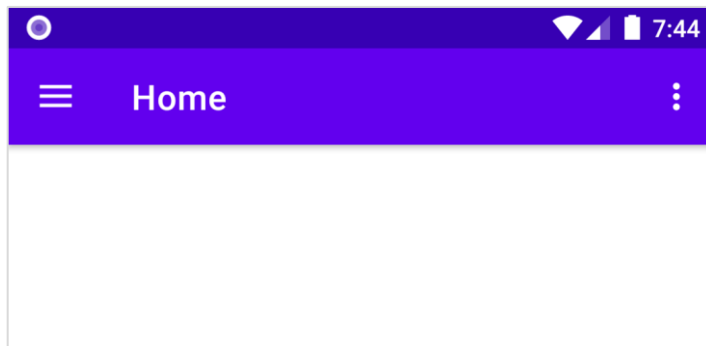
**Barra superior  
(Appbar)**

# Barra de la app

---

La barra superior o *barra de acciones* (*ActionBar*) proporciona una estructura visual coherente con otras apps, haciendo comprensible su manejo:

- Proporciona una identidad a tu app e indica la ubicación del usuario en ella.
- Ofrece acceso predecible a acciones importantes, como la búsqueda.
- Compatible con la navegación mediante botones y menús.



# Definir un menú

---

Crear un nuevo .xml de tipo Menu desde *res/* que contenga los elementos de nuestro menu.

- Definir para cada item: id, icono y título.
- Elegir cuáles mostrar en función del espacio ajustando su *app:showAsAction*

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

  <item
    android:id="@+id/action_gallery"
    android:icon="@android:drawable/ic_menu_gallery"
    android:title="@string/menu_gallery"
    app:showAsAction="ifRoom"/>
  <item
    android:id="@+id/action_settings"
    android:orderInCategory="100"
    android:title="@string/action_settings"
    android:icon="@android:drawable/ic_menu_preferences"
    app:showAsAction="never" />
</menu>
```

menu\_toolbar.xml

# Añadir la *appbar* al diseño de la actividad

Para añadir la barra superior de navegación a la actividad, en el .xml correspondiente se añade un *AppBarLayout* que contenga una *MaterialToolbar*.

Además, hay que asegurarse de que el *Theme* de la aplicación definido en el *AndroidManifest.xml* termina en *NoActionBar*.

```
<com.google.android.material.appbar.AppBarLayout
    android:id="@+id/appbarlayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:layout_constraintTop_toTopOf="parent">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:title="Home"
        app:navigationIcon="@drawable/ic_menu"
        style="@style/Widget.MaterialComponents.Toolbar.Primary" />

</com.google.android.material.appbar.AppBarLayout>
```

activity\_main.kt

Icono opcional. Si no se especifica, se pone la flecha de ir atrás

# Inicializar la *appbar* en la actividad

---

Como con el resto de elementos de diseño que queremos gestionar, en la actividad se debe crear un atributo y asociarlo a su vista. Además:

1. Asignar la *toolbar* como barra de acción via *setSupportActionBar()*.
2. Sobreescibir *onCreateOptionsMenu* para inflar el menú antes creado.

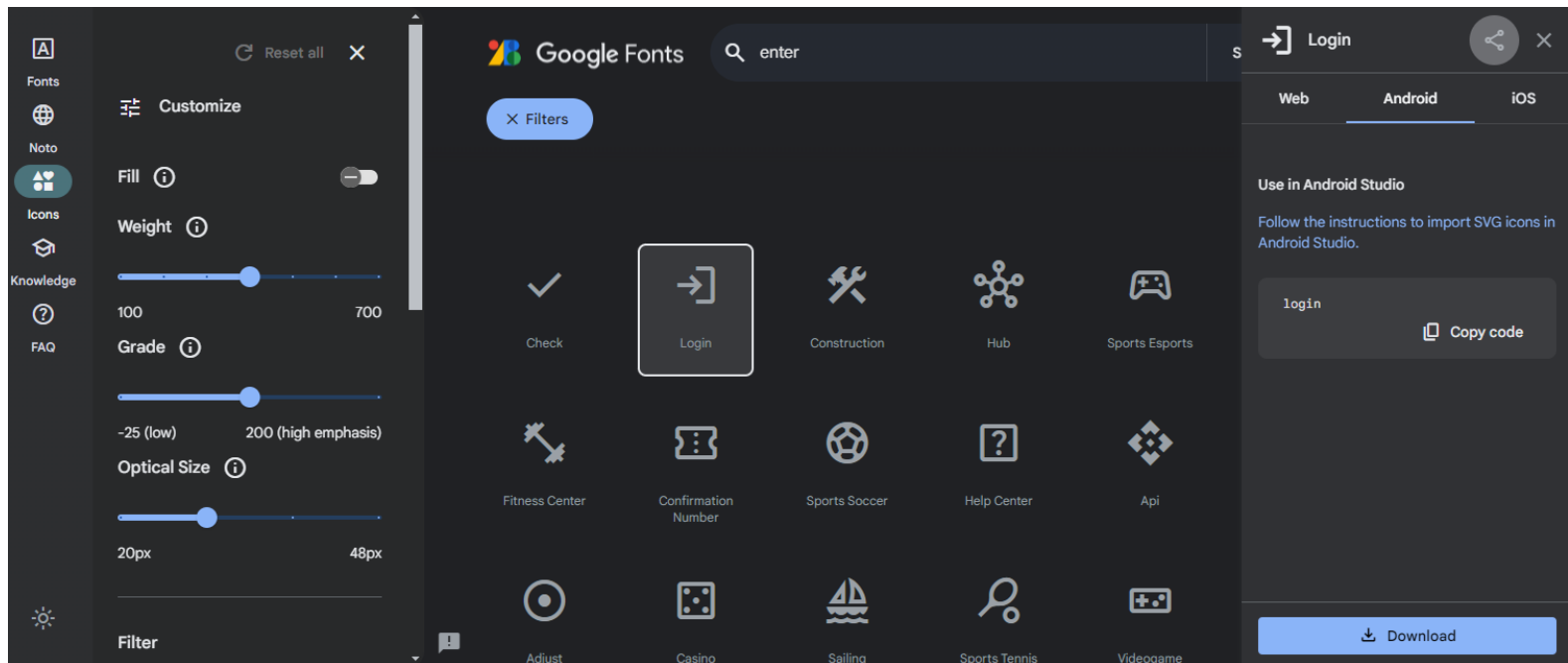
```
class MainActivity : AppCompatActivity() {  
    private lateinit var toolbar: Toolbar  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        toolbar = findViewById(R.id.toolbar)  
        setSupportActionBar(toolbar)  
    }  
  
    override fun onCreateOptionsMenu(menu: Menu?): Boolean {  
        menuInflater.inflate(R.menu.menu_toolbar, menu)  
        return true  
    }  
}
```

MainActivity.kt

# Iconos de *Material Design*

Material Design es la guía de diseño estándar en Android. Para usar sus iconos:

1. Ir a <https://fonts.google.com/icons?icon.platform=android>
2. Buscar y seleccionar el icono deseado
3. Descargar el archivo .xml de Android y pegarlo en *res/drawable/*



# Responder a interacciones

---

Para capturar las interacciones con los elementos del menú de la barra superior se debe sobrescribir el método *onOptionsItemSelected*.

Recibe como parámetro un MenuItem, cuyo atributo itemId nos permite saber qué opción del menú ha sido clickada por el usuario.

```
class MainActivity : AppCompatActivity() {  
    override fun onOptionsItemSelected(item: MenuItem): Boolean {  
        when(item.itemId){  
            R.id.action_checkout -> Log.d("Toolbar", "Acción de la galería")  
            R.id.action_account -> Log.d("Toolbar", "Acción del carousel")  
            R.id.action_settings -> Log.d("Toolbar", "Acción de las preferencias")  
            else -> super.onOptionsItemSelected(item)  
        }  
        return true  
    }  
}
```

MainActivity.kt

# Ejercicio 1

---

Siguiendo con el ejercicio de la app de contactos de la sesión anterior:

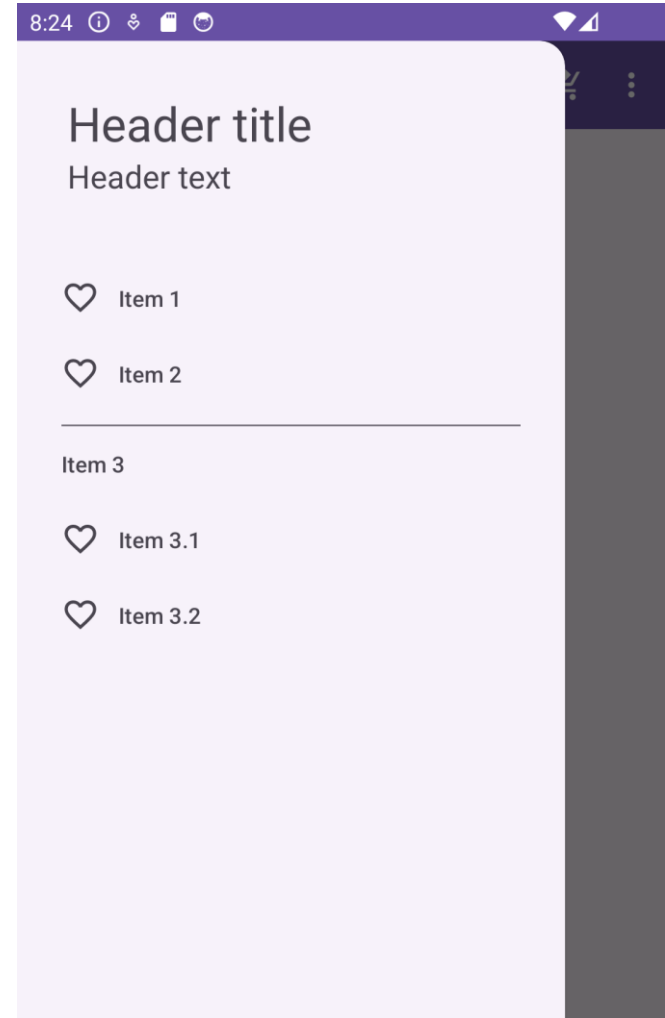
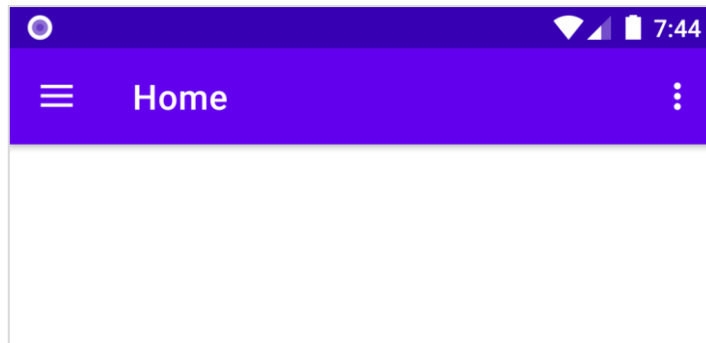
1. Realiza las modificaciones necesarias para añadir una barra de acciones a la aplicación.
2. Añade a la barra un menú con dos acciones:
  1. La primera: con el icono de Delete Sweep de la colección Material, que vacíe la lista de contactos. El icono se debe mostrar en la barra.
  2. La segunda: con el icono Info, que esté escondida en el menú desplegable, que muestre un Toast con el nombre del autor de la aplicación.

# **Menú lateral (Navigation Drawer)**

# Menú lateral

En Android, es común que las aplicaciones desplieguen un menú lateral que nos permita navegar a otras pantallas, gestionar nuestra cuenta o administrar las preferencias...

Ese menú se conoce como *Navigation Drawer* y está accesible desde el botón *Home* situado en la parte izquierda de la *toolbar*.



# Definir un menú

menu\_drawer.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
  <group android:checkableBehavior="single">
    <item
      android:id="@+id/nav_gallery"
      android:icon="@android:drawable/ic_menu_gallery"
      android:title="@string/menu_gallery"/>
    <item
      android:id="@+id/nav_slideshow"
      android:icon="@android:drawable/ic_menu_slideshow"
      android:title="@string/menu_slideshow"/>
  </group>
  <group
    android:id="@+id/group1"
    android:checkableBehavior="single">
    <item
      android:id="@+id/nav_settings"
      android:title="@string/action_settings"
      android:icon="@drawable/ic_settings"/>
  </group>
</menu>
```

Los grupos permiten crear agrupamientos entre opciones del menú relacionadas. Además, los grupos quedan separados visualmente por líneas divisoras

# Añadir el menú lateral al diseño de la actividad

---

El diseño de una actividad con menú lateral debe tener un *DrawerLayout* como ViewGroup raíz. Dentro, , se añadirá un *NavigationView*, que será el elemento responsable de mostrar el menú.

Si queremos añadir más elementos de diseño, deberán ir en un *ConstraintLayout* al mismo nivel que el *NavigationView*.

```
<androidx.drawerlayout.widget.DrawerLayout
...
  android:id="@+id/drawer_layout"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:openDrawer="start">

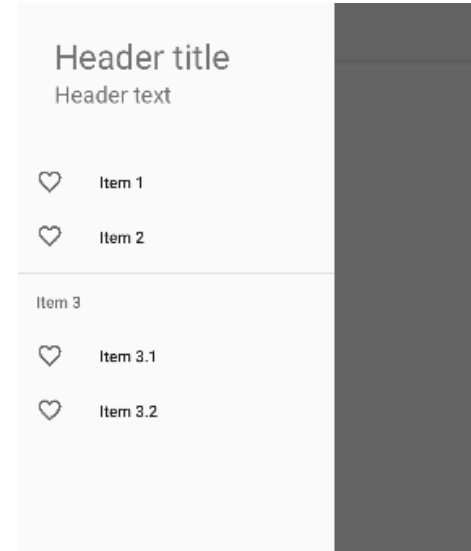
  <com.google.android.material.navigation.NavigationView
    android:id="@+id/nav_view"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:layout_gravity="start"
    app:headerLayout="@layout/nav_header_main"
    app:menu="@menu/menu_drawer"/>
</androidx.drawerlayout.widget.DrawerLayout>
```

activity\_main.xml

# Reestructurar el diseño con *include*

En Android Studio es muy incómodo trabajar con los diseños basados en un `DrawerLayout`, puesto que el resto de elementos de la interfaz no se muestran en la vista previa.

Para solucionarlo, es recomendable mover el diseño de la actividad a otro `.xml`, e incluir este archivo al mismo nivel que el `NavigationView`.



```
<androidx.drawerlayout.widget.DrawerLayout
...
<include layout="@layout/content_main"/>

<com.google.android.material.navigation.NavigationView
... />
</androidx.drawerlayout.widget.DrawerLayout>
```

activity\_main.xml

En `content_main.xml` se define el layout con los elementos de UI necesarios

# Inicializar el menú en la actividad

Como con el resto de elementos de la UI, en la actividad creamos variables y las asociamos a sus vistas correspondientes. Además, mostramos en la appbar el botón de navegación hacia arriba llamando a *setDisplayHomeAsUpEnabled*.

Luego, configuramos la appbar para que el botón Home muestre el menú lateral.

```
class MainActivity : AppCompatActivity() {
    private lateinit var toolbar: Toolbar
    private lateinit var drawerLayout: DrawerLayout
    private lateinit var navView: NavigationView
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main_drawer)
        toolbar = findViewById(R.id.toolbar)
        setSupportActionBar(toolbar)
        supportActionBar?.setDisplayHomeAsUpEnabled(true)
        drawerLayout = findViewById(R.id.drawer_layout)
        navView = findViewById(R.id.nav_view)
    }
}
```

MainActivity.kt

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    when(item.itemId){
        android.R.id.home -> drawerLayout.open()
        // Resto de opciones del menú ...
    }
    return true
}
```

# Responder a interacciones

Para asociar acciones a cada elemento del menú lateral, le asignamos un callback al `NavigationView` via `setNavigationItemSelectedListener`.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        // A continuación de lo anterior  
        navView.setNavigationItemSelectedListener {  
            drawerLayout.close()  
            when(it.itemId){  
                R.id.nav_gallery -> Log.d("Drawer", "Acción de la galería")  
                R.id.nav_slideshow -> Log.d("Drawer", "Acción del carrousel")  
                R.id.nav_settings -> Log.d("Drawer", "Acción de las preferencias")  
            }  
            true  
        }  
    }  
}
```

MainActivity.kt

Usando el objeto `DrawerLayout`  
cerramos el menú

# Ejercicio 2

---

Siguiendo con el ejercicio anterior:

1. Añade un menú lateral a la actividad principal que contenga dos grupos con dos y tres items, respectivamente.
2. Asocia un callback que muestre en el Logcat un mensaje diferente cuando se pulse cada uno de los elementos del menú.

# **Botón de acción flotante (FAB)**

# Añadir un FAB a la actividad

---

Un botón de acción flotante (Floating Action Button) es un patrón de diseño muy común en Android.

Se trata de un botón circular en la esquina inferior derecha que se superpone a la vista de la actividad y que habilita la acción principal de la interfaz, que no se puede realizar mediante ningún otro elemento de la misma.

```
<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fab"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintBottom_toBottomOf="parent"
    android:src="@drawable/ic_launcher_foreground"
    android:contentDescription="Añadir un contacto"
    android:layout_margin="16dp" />
```

content\_main.xml

# Asignar una acción al evento onClick

---

Una vez añadida la vista del FAB al diseño de la actividad, se debe implementar su funcionalidad en el *onCreate* de la Actividad correspondiente.

Se le asigna un *onClickListener* de la misma forma que con otros elementos.

MainActivity.kt

```
class MainActivity : AppCompatActivity() {  
    private lateinit var fab: FloatingActionButton  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        fab = findViewById(R.id.fab)  
        fab.setOnClickListener(){  
            Toast.makeText(this, "FAB", Toast.LENGTH_SHORT).show()  
        }  
    }  
}
```

# Ejercicio 3

---

Siguiendo con el ejercicio anterior:

1. Añade un FAB a la actividad principal con el icono Material: *Person Add*.
2. Crea una nueva actividad que permita introducir la información de contacto que antes se captaba en la parte inferior de la actividad principal. Añade un botón abajo con el texto CREAR CONTACTO.
3. Modifica el diseño de la actividad principal para que el RecyclerView ocupe todo el espacio de la pantalla, quitando los campos de abajo.
4. Haz que el FAB lance la actividad para crear un contacto.
5. En la actividad de creación de contacto, implementa el evento *onClick* del botón para que al pulsarlo se envíen de vuelta a la actividad principal.
6. En la actividad principal, actualizar la lista con el nuevo contacto creado cuando se reciban los datos.

# Referencias

---

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>